

From Pixels to Points

AI-Powered Tennis Match Insights

Marlon Helbing

marlonjoshua.helbing@studenti.unipd.it

Daniele Virzi

daniele.virzi@studenti.unipd.it

Alberto Calabrese

alberto.calabrese.2@studenti.unipd.it

Abstract

This report presents a comprehensive suite of computer vision-based analysis tools for detailed tennis match analysis. Our system is designed to address key components of gameplay analysis, including the detection and tracking of players and the ball, identification of racket hits and ball bounces, as well as dynamic court line detection. Finally, we predict the player's score probabilities based on a ball & player-based heatmap. In particular, we introduce a deep learning method for ball bounce detection leveraging a lightweight CNN and DBSCAN, achieving 84% in detecting true bounces within 2 frames. Additionally, an audio-visual approach for racket hit detection is proposed, demonstrating 84.74% accuracy within a 6 frame range.

1. Introduction

Tennis is the world's fourth most-watched sport, attracting roughly one billion viewers worldwide. Although live matches are exciting to watch, analyzing the game afterwards can reveal interesting patterns for viewers and give players useful feedback on their performance. With this project, we developed a comprehensive suite of analysis tools designed to extract detailed insights from tennis videos. Our system can identify and follow players throughout a match, track the ball's movement, recognize the court layout, detect key events like when a player hits the ball or when the ball bounces on the court and provide score probabilities for the players during the rallies. We made several key contributions to tennis analysis: First, we created a new method for detecting ball bounces that treats it as a visual pattern recognition problem. Our approach uses a lightweight CNN which achieves an F1-score of 81.25%, and combined with DBSCAN [3] clustering is able to detect true bounces within 2 frames with an accuracy of 84.0%. Second, we developed a new way to detect racket hits by combining both audio signals and visual cues, which

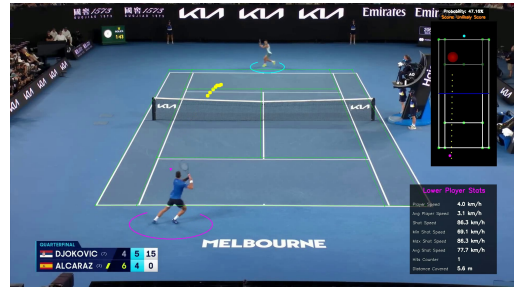


Figure 1. Final output of our match analysis system.

gives us a 84.74% accuracy of detecting a racket hit within 6 frames. Finally, we built a novel method to compute the score probability of players based on their position and the estimated ball landing position. To present our findings in a clear and visually engaging way (1), we leveraged modern computer vision libraries. Our work represents a step forward in automated sports analysis, providing tools that can give viewers and analysts much deeper insights into tennis matches.

2. Related Work

Huang et al. [5] introduced TrackNet, a deep learning network for tracking high-speed objects in sports, which achieved a 98.5% F1-score by using CNNs to generate ball location heatmaps from consecutive frames. In our work, we adopt this approach to track tennis balls. Existing ball bounce detection approaches employ typical Machine Learning approaches, such as the Time Series Forest Classifier utilized in the ArtLabs tennis tracking system [1]. While temporal modeling intuitively captures the sequential nature of bounce events, we propose that bounce detection can be reformulated as a visual pattern recognition problem. Specifically, the distinctive V-shaped trajectory pattern created during ground contact presents a compelling case for CNN-based classification. Current research in tennis analytics primarily focuses on predicting overall match

outcomes or winners. These methods, like the one presented in [9], mostly use machine learning with tabular data. This data typically includes aggregated information such as player rankings, past performance, and various match statistics. While these approaches effectively forecast high-level results, they can't analyze the dynamic changes happening during a match. Our new method fills this gap by introducing a computer vision based approach. This allows us to predict shot-by-shot score probabilities based on the video of a tennis match. It generates predictions about a player's chance of winning the current point, based on the immediate context of each shot.

3. Dataset

3.1. Tennis Matches

We extracted 40 video segments ranging from 4 to 20 seconds from the Australian Open 2025 quarter-final match between Djokovic and Alcaraz. The videos show single tennis rallies filmed from a tilted bird's-eye view. The videos were chosen such that they capture diverse scenarios including forward runs, short balls, and typical ball exchanges.

3.2. Courtline

The dataset consists of 8,841 images with a resolution of $1,280 \times 720$ pixels, encompassing all three court types including hard, clay, and grass surfaces. The images were downloaded from a GitHub Repository [7]. Each image contains 14 annotated keypoints, and the dataset was divided into training and testing sets using a 75 - 25 split respectively. Since we will leverage a ResNet-34 [4] architecture pretrained on ImageNet, the images were rescaled to 224×224 , with coordinate labels scaled proportionally and images normalized using ImageNet's mean and standard deviation values across RGB channels.

3.3. Ball Bounces

Given the novelty of our approach using Deep Learning, we developed a custom dataset for bounce detection. The problem was formulated as a binary classification task distinguishing between bounce and non-bounce events. From the dataset mentioned in 3.1, we extracted bounce frames for each video.

3.3.1 V-Pattern Visualization and Labelling

We rendered the ball's trajectory across 6 frames, meaning that for any current frame k , the 5 previous ball positions are also rendered. This create a distinctive V-pattern at bounce events, where the downward path sharply transitions upward upon ground contact. Non-bounce trajectories remain smooth without these characteristic angular changes (Figs. 2). To ensure clear V-pattern visibility, we labeled



Figure 2. Example of a bounce (left) and no bounce (right) pattern. Notice the V-shape a bounce creates.

frames $k + 2$, $k + 3$ and $k + 4$ as bounce examples for each actual bounce at frame k . Frames $k + 1$ and $k + 5$ were excluded from the dataset due to potential bounce/non-bounce ambiguity.

3.3.2 Data Quality, Balance and Efficiency

To address tracking artifacts directly after racket hits that mixed pre- and post-hit information - often looking similar to a V-pattern - we reset the trace at every detected racket hit. Due to noticeable tracking problems from high ball speed after serves, we remove all frames until the opponent's response hit. To ensure strong representation for our bounce class, any bounce example triplet ($k + 2$, $k + 3$, $k + 4$) containing missing ball tracking data - and therefore representing incomplete visual patterns - was entirely removed from the dataset. To address the data imbalance, we doubled bounce examples using vertical flipping augmentation. Horizontal flipping was avoided as it would create unrealistic distortions, given the tilted bird's-eye perspective. Finally, to reduce computational complexity, we filtered images to show only the ball trace, converted them to grayscale, and resized them to 128×128 .

3.3.3 Data Split

To prevent data leakage, we employed video-level splitting where different videos were allocated to training, validation and test sets. This approach ensures that temporally adjacent frames from the same video sequence do not appear across different dataset partitions. The dataset was split using a 80/10/10 ratio. Note that the previously mentioned augmentation method was only applied to the training set. All images were normalized using mean and standard deviation values calculated on the training set.

4. Method

4.1. Player Detection & Tracking

4.1.1 Person Detection & Tracking

For each frame in the video sequence, we implement object detection and tracking using YOLOv8 [6]. This system detects objects and assigns persistent IDs maintained across frames, enabling us to track individual players throughout the video sequence. Since we are interested in tracking the

two players, we apply a filtering criteria to retain only those detected objects classified as 'person' category.

4.1.2 Players Selection

To isolate the two players from other detected individuals, we operate on the assumption that the players are the most mobile. The process begins by tracking the center of each person's bounding box and immediately discarding anyone detected in fewer than five frames.

For each remaining candidate, we calculate a weighted score $D_{weighted}$ that combines motion and presence. This score multiplies the person's average displacement per frame (D_{avg}) by their detection consistency (C) — the fraction of frames in which they appear. As shown in the equation (1), this metric effectively represents an individual's total distance traveled normalized by the total number of frames in the sequence. The two people with the highest weighted scores are selected as the players, and all other detections are filtered out.

- d_i : The position vector (e.g., (x_i, y_i)) at frame i .
- n : The total number of recorded movements for the person.
- N_{total} : The total number of frames where any person was detected.

$$\begin{aligned}
 D_{weighted} &= D_{avg} \cdot C \\
 &= \frac{D_{total}}{n} \cdot \frac{n}{N_{total}} \\
 &= \frac{1}{n} \sum_{i=1}^{n-1} \|d_i - d_{i-1}\| \cdot \frac{n}{N_{total}} \quad (1) \\
 &= \frac{1}{N_{total}} \sum_{i=1}^{n-1} \|d_i - d_{i-1}\|
 \end{aligned}$$

4.2. Court Keypoint Detection

Our system leverages 14 predefined keypoints, which correspond to critical court intersections such as baseline corners, service line intersections, center marks, and net line crossings. Once we obtain from the ResNet-34 [4] model the prediction of the 14 keypoints, a post-processing pipeline is applied to enhance the accuracy and stability of the detected keypoints. The process starts by scaling the coordinates from the model's 224×224 resolution to the original frame size, then it applies a refinement algorithm to correct minor errors. This algorithm works by performing local averaging for each predicted keypoint, identifying a cluster of neighbor keypoints within a small Euclidean distance (20 pixels). The refined position is then computed as the average coordinate of this cluster, leading to a more stable and geometrically coherent set of court landmarks. To enhance

robustness against camera movements and zoom variations and create a visually precise final output, we implemented a dynamic mode that predicts the keypoints independently on every frame, following the GitHub repository. [7].

4.3. Ball Detection & Tracking : TrackNet

TrackNet is a CNN that takes 3 consecutive frames as input and outputs a heatmap indicating where the ball is likely located [5]. It's designed specifically for small, fast-moving objects like tennis balls. We utilize a pre-trained network found on the Git Repository [8] and will be using this as our ball tracking framework from here on.

4.4. Racket Hit Detection

We aim to detect when a tennis player strikes the ball with their racket during gameplay. Two primary approaches are explored to address this challenge. The first approach leverages audio analysis by identifying the distinctive loud sounds produced during racket-ball contact. While racket hits typically generate detectable audio peaks, this method is susceptible to false positives from player movement and crowd noise. The second approach tracks directional changes in the tennis ball's trajectory, based on the principle that the ball only changes direction upon racket contact. However, this method requires precise ball tracking and is complicated by the tilted bird's-eye camera perspective, which introduces coordinate uncertainties. Additionally, this approach cannot detect serves due to the absence of directional change. Since both methods offer complementary strengths, we propose an **audio-visual** approach that combines them: audio provides precise frame-level timing but generates false positives, while visual tracking gives approximate hit locations. By identifying instances where audio peaks and visual directional changes occur in close temporal proximity, we achieve a higher confidence hit detection.

4.4.1 Audio Signal

First, we apply frequency filtering using a 6th-order Butterworth filter (150 – 1800Hz, normalized by Nyquist frequency). This empirically determined range preserves the characteristic "thump" of racket hits while filtering out high-pitched sounds and low-frequency interference from crowd noise and footsteps. Next, the filtered audio waveform is converted to signal energy over time. The absolute value of the signal is computed to convert all amplitude values to positive, enabling straightforward peak detection. The resulting signal is then smoothed using a moving average filter with a 10 ms window size, with zero-padding applied at signal boundaries. Finally, peak detection is performed using three key parameters (Fig. 3). The *height* threshold is set to 0.01 to ensure detection of all racket hits

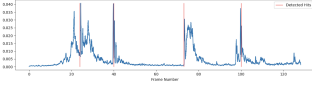


Figure 3. Peak detection in audio signals related to tennis matches.

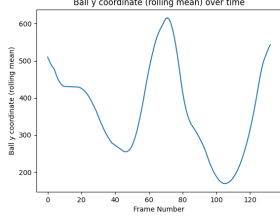


Figure 4. Directional change detection in visual signals related to tennis matches.

(i.e. soft and hard). A minimum *distance* constraint of 0.25 seconds between peaks filters out vocal expressions such as shouts that commonly follow racket strikes. A *prominence* threshold of 0.006 ensures detected peaks represent genuine acoustic events rather than minor signal fluctuations.

4.4.2 Visual Signal

To reduce tracking noise, the ball tracking coordinates are first smoothed using a rolling average with a window size of $0.4 \times fps$ frames (Fig. 4). This 0.4-second window accounts for the typical duration of a direction change event. Direction change detection uses a two-stage validation process. First, the algorithm identifies potential local maxima or minima by comparing each smoothed ball coordinate with its direct neighbors, finding candidates where preceding and following coordinates are consistently higher or lower. To reduce false positives from noisy tracking, a temporal consistency check verifies that all coordinates within the subsequent 0.2 seconds maintain the expected relationship to the extremum value. This filters out artificial oscillations from tracking errors while preserving legitimate rapid directional changes. A direction change is confirmed only when both the local extremum condition and the temporal consistency requirement are satisfied.

4.4.3 Audio-Visual approach

We define that the audio and visual signals of a possible racket hit can only have a maximum temporal separation of $0.5 \times fps$ frames, which we empirically determined to provide optimal performance in our work. For each detected visual hit, all audio hits within the specified temporal window are identified, creating tuples of (*visual_frame*, *audio_frame*) that represent potential matches. These tuples are then clustered by court side, with each cluster representing potential racket hits occurring on the same side of the court. Finally, for each group of potential racket hits,

the algorithm identifies the most probable hit by selecting the tuple whose audio frame corresponds to the ball position closest to the player using Euclidean distance. Since serves produce distinctive acoustic signals but lack directional changes for visual detection, we apply only our audio peak detection with an elevated *prominence* threshold of 0.01 and select the first detected peak as the serve.

4.5. Ball Bounce Detection

4.5.1 Model

We present a lightweight CNN (Fig. 5) that takes as input the Bounce Dataset as described in Section 3.3 and outputs in which frames it found a V-Pattern. The compact architecture was chosen to prevent overfitting on our small dataset of sparse images, which contain minimal visual information within largely empty regions. While we addressed some of the data imbalance issues through our augmentation strategy, our dataset still has substantially more non-bounce examples compared to bounce examples. To appropriately evaluate model performance under these conditions, we calculate precision, recall and F1-score.

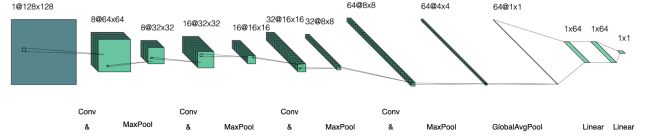


Figure 5. CNN architecture for bounce detection.

4.5.2 Clustering

Our model predicts multiple frames where V-shaped patterns, indicative of a ball bounce, are clearly visible. Consequently, the actual bounce event occurs several frames prior to these visual cues. To identify the ball bounce frame from these predictions, we employed an unsupervised approach. Specifically, we implemented DBSCAN [3], with hyperparameters $\epsilon = 3$ and $MinPts=2$, to cluster all predicted frames. Each resulting cluster represents a distinct bounce event. From each cluster, we then selected the initial frame and subtracted a constant delay, δ . We set $\delta = 2$ based on our dataset annotation strategy, where frames $k+2$, $k+3$, and $k+4$ serve as bounce examples while the actual bounce occurs at frame k .

4.6. Mini-Court Visualization

We created a 2D mini-court representation of the playing area in order to extract scoring probability and improve detection visualization. This was accomplished by transforming the court from the viewpoint of the camera to a top-down, planar view using a homographic matrix transformation [2]. To guarantee precise spatial scaling, a con-

version rate between pixels and meters was applied. Thanks to this representation, we were able to overlay players and ball detected positions accurately and trajectories.

4.7. Player Statistics

This system analyzes tennis rallies by first identifying the serving player. When the first racket hits the ball, the system assigns the serve to the player closest to the ball. This player is labeled the server, and their opponent is the receiver. Throughout the match, the system uses the players' y-positions to continuously identify them as the upper or lower player. After these labels are assigned, the system compiles and displays statistics specific to the player selected at the beginning of the analysis. A triangle is placed over the selected player's head for the duration of the analysis, and their identity is clearly indicated in the statistics display. In the context of this report, the player who is not selected for statistical analysis will be referred to as the "opponent". Once players are identified and labeled, this information serves as the foundation for presenting performance insights. To this end, we designed a dynamic overlay that displays statistics for the selected player during video playback. Its primary purpose is to offer an intuitive and continuously updated summary of a player's performance as the match unfolds. The system displays key performance indicators such as player speed and shot speed. In addition, the system supports a two-player mode, which is useful for direct performance comparison. The Shot Speed (v_{shot}) is computed as the ratio between the Euclidean distance covered by the ball and the time between the hit and its subsequent bounce $v_{\text{shot}} = d_{\text{hit, bounce}} / t_{\text{hit, bounce}}$ where $d_{\text{hit, bounce}}$ represents the Euclidean distance between the ball's position at the moment of impact and its landing position. This distance, initially in pixels, is converted to meters using a known scaling factor derived from the court dimensions $d_{\text{meters}} = d_{\text{pixels}} \cdot \frac{h_{\text{meters court}}}{h_{\text{pixels court}}}$. The term $t_{\text{hit, bounce}}$ denotes the time difference between these events, directly derived from the frame difference and the video's frame rate ($t_{\text{hit, bounce}} = \frac{\Delta_{\text{frame}}}{\text{FPS}}$). Player Speed (v_{player}) is continuously updated by computing the player's displacement between consecutive frames within the mini-court reference system. The actual player speed for a given frame is determined by: $v_{\text{player}} = d_{\text{frame-to-frame}} / t_{\text{frame}}$ where $d_{\text{frame-to-frame}}$ is the displacement between the player's positions in consecutive frames, and $t_{\text{frame}} = \frac{1}{\text{FPS}}$ is the time duration of a single frame. Finally, the cumulative distance (D_{total}) covered by the player during the match is obtained by summing all frame-to-frame displacements.

4.8. Score probability

To model the winning probability of a shot, we assumed that the farther the opponent player is from the ball's estimated landing position, the lower their chance of returning

the shot. To quantify this, we created two heatmaps: one based on the receiving player's position and another for the estimated ball landing position. By combining these two heatmaps, we were able to derive a probability of scoring.

4.8.1 Player Heatmap

We generated a heatmap for the opponent's mini-court by mapping each zone based on its L2 distance from the receiving player's position. The values of the heatmap were normalized to a range of 0 to 1, where 0 represents the opponent player's exact location and 1 corresponds to a distance equal to the diagonal of the opponent's mini-court. This process was executed efficiently at each frame using NumPy vectorization to ensure scalability.

4.8.2 Ball Heatmap

Based on the detected ball's position at the predicted bounce frame, we can estimate its probable landing zone on the opponent's mini-court. To account for potential estimation errors, we defined a circular zone with a radius of $\rho = 20\text{px}$ around the estimated landing spot. Within this zone, the estimated position itself is assigned a value of 1, with surrounding areas receiving values between 0 and 1 based on their L2 distance from the center. Any area of the court outside this circular zone is given a value of 0, as it's highly improbable the ball will land there.

4.8.3 Score Modeling

The final score probability is derived from a heatmap (Fig. 6) that combines the ball's predicted landing position with the opponent's location. The maximum value on this map indicates the likelihood of scoring a point. Initial testing revealed a bias where this method produced unrealistically low probabilities, even for clear scoring shots. To correct this, we adopted an adjusted formula. The final probability is now calculated as a weighted sum of the player's heatmap (H_{player}) and the combined heatmap (H_{combined}), giving greater influence to the player's position. The result is clipped at 1 to ensure a valid probability, yielding a more accurate scoring model. Here (2), we set the weights to $\alpha = 0.15$ and $\gamma = 1$ to correct the observed bias.

$$\mathbb{P}(\text{score}) = \text{clip}(\max(\alpha H_{\text{player}} + \gamma H_{\text{combined}}), 1) \quad (2)$$

5. Experiments

5.1. Prediction of Courtline

We employed transfer learning by fine-tuning a pre-trained ResNet-34 model that was originally trained on ImageNet. The final classification layer was replaced with a

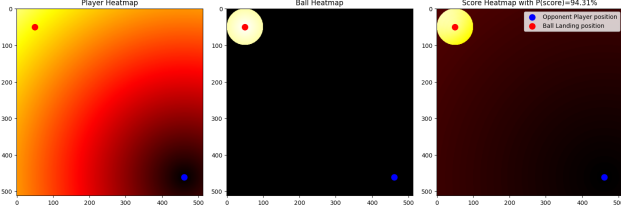


Figure 6. Heatmap logic to extract score probability.

linear layer featuring linear activation that outputs 28 values, corresponding to the x and y coordinates of the 14 key-points. The model was trained over 20 epochs using a batch size of 16, employing the Adam optimizer with a learning rate of $1 \cdot 10^{-4}$ and MSE loss. We achieved a total loss of 1.16 on the training set and 9.26 on the validation set.

5.2. Prediction of Racket Hits

Across the 40 video clips (3.1), our model achieved 84.74% accuracy in predicting racket hits within 6 frames of the ground truth and 72.11% accuracy within a 2 frame tolerance. In 22.86% of the videos, some actual hits were entirely missed by our model and in 11.43% of the videos, our model detected false positive hits.

5.3. Prediction of Ball Bounces

Our CNN uses the AdamW optimizer (initial LR 1×10^{-4} , weight decay 1×10^{-6}), a LR scheduler (patience 3, reduction factor 0.5) and is trained for 50 epochs with a batch size of 32 and early stopping based on validation loss. To address class imbalance, we use weighted binary cross entropy loss with positive weight calculated as the non-bounce to bounce ratio, up-weighting the minority bounce class. During training, we apply data augmentation of rotations up to 10 degrees. The results of our model can be seen in Table (1). We conducted additional experiments by rendering the ball’s trajectory across 3 and 10 frames but found that 6 provided optimal performance. Furthermore, we explored incorporating court lines into the images to provide additional spatial context for the CNN, but this approach yielded no measurable improvement in classification accuracy. To compare our CNN bounce detection model to ArtLabs Forest Classifier [1], we use the clustering approach as described in 4.5.2 to get exact frame predictions. However, it is important to note that each model was trained and evaluated on different datasets with possibly varying pre-processing methods. In particular, our model doesn’t detect the bounce following the serve (as this event was excluded from the dataset) and is not trained on detecting the exact frame of a bounce, but rather in finding the area where a bounce occurs. We therefore evaluate not on exact frames, but using a tolerance level of 1 and 2 around the exact frame prediction and do not take into consideration the post-serve bounce. The results can be seen in 2).

Precision	Recall	F1 Score
0.7429	0.8966	0.8125

Table 1. Bounce detection model results.

Model	Recall	Specificity
CNN (ours)	0.90	0.95
CNN (ours, exact frame ± 1)	0.79	0.99
CNN (ours, exact frame ± 2)	0.84	0.99
Forest Classifier (ArtLabs)	0.83	0.98

Table 2. Bounce detection model comparison. Recall refers to accuracy of predicting true bounces, while Specificity refers to accuracy of predicting true non bounces.

6. Conclusion

With this project, we successfully developed a comprehensive computer vision system for automated tennis match analysis. Our application allows player & ball tracking and court keypoint, ball bounce and racket hit detection alongside a player scoring probability prediction. The potential applications of this technology extend beyond tennis analysis to other racket-based sports and could be adapted for coaching applications, broadcast enhancement, and player performance evaluation. However, our work also revealed several limitations and opportunities for future improvement. The ball bounce detection system processes largely sparse 128×128 grayscale images with minimal visual information. A promising direction would be isolating only the ball trace pixels, effectively eliminating the sparse background information that may negatively impact performance. Furthermore, our current approach treats bounce detection as a static pattern recognition problem, but incorporating temporal dynamics could enhance robustness. Following TrackNet’s [5] methodology of using 3 consecutive frames as input, our CNN could learn to detect not just the V-pattern itself, but the sequential “creation” of the pattern. Our racket hit detection algorithm faces challenges with temporal precision. The relatively wide tolerance window of 6 frames means our system often associates hits with nearby but incorrect audio events such as shoe sliding or player grunting, which occur in close temporal proximity to actual racket contact. A potential solution would be implementing more sophisticated audio filtering techniques to eliminate these sounds, such as CNNs for audio classification. Additionally, imprecise ball tracking can cause failure of our euclidean distance-based selection approach. Ultimately, the score model’s performance hinges on accurately detecting every racket hit and ball bounce. While small frame detection errors are acceptable, missing any event compromises functionality. A key failure arises with volleys, which, despite being rare at the professional level today [10], are a valid play in tennis.

References

- [1] ArtLabss. Open-source monocular python hawkeye for tennis, 2021. GitHub Repository.
- [2] Ondřej Chum, Tomáš Pajdla, and Peter Sturm. The geometric error for homographies. *Computer Vision and Image Understanding*, 97(1):86–102, 2005.
- [3] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, KDD’96, page 226–231. AAAI Press, 1996.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.
- [5] Yu-Chuan Huang, I-No Liao, Ching-Hsuan Chen, Tsì-Uí Ík, and Wen-Chih Peng. Tracknet: A deep learning network for tracking high-speed and tiny objects in sports applications, 2019.
- [6] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8, 2023.
- [7] Sergey Kosolapov. Tennis court detector: Deep learning network for detecting tennis court, 2023. GitHub Repository.
- [8] Sergey Kosolapov. Tracknet: Unofficial pytorch implementation of tracknet, 2023. GitHub Repository - Unofficial PyTorch implementation.
- [9] Yilin Lei, Ao Lin, and Jianuo Cao. Rhythms of victory: Predicting professional tennis matches using machine learning. *IEEE Access*, 12:113608–113617, 2024.
- [10] Wikipedia. Serve-and-volley — Wikipedia, the free encyclopedia. <http://en.wikipedia.org/w/index.php?title=Serve-and-volley&oldid=1269422102>, 2025.